Tokenization for Large Language Models

Summer 2026, LMU Munich

Prof. Dr. Valentin Hofmann

Seminar Information

- This seminar is about tokenization for large language models (LLMs)
- Tokenization is a highly multi-faceted topic with connections to:
  - Linguistics (especially morphology) and cognitive science
  - Deep learning theory
  - Fairness and bias
  - Model robustness
  - …
- We will learn about tokenization basics and explore some more advanced issues in greater detail

Seminar Logistics

- There will be two seminar phases
- Phase 1:
  - 17.04.2026, 24.04.2026, 08.05.2026
  - Introduction to tokenization
  - Assignment of topics
- Phase 2:
  - Two-day block shortly after the end of the semester
  - Proposed dates: 20.07.2026, 21.07.2026
  - Student-led presentations

Details About Assessment

- For MSc students in Computational Linguistics: pass/fail
- Presentation (ca. 20 minutes) plus term paper (ca. 25.000 characters)

Questions?

Roadmap for Today

Seminar Introduction ✔

What Is Tokenization, and Why Do LLMs Need It?

LLM Recap (Whiteboard)

Roadmap for Today

Seminar Introduction ✔

What Is Tokenization, and Why Do LLMs Need It?

LLM Recap (Whiteboard)

How Do LLMs See Text?

- An LLM cannot simply digest the input text as is (infinite number of possible input texts)
- We need to map the input text to a sequence of units that the LLM “knows”
- This step is commonly referred to as tokenization, and the units known to the LLM as tokens

How Do LLMs See Text?

- An LLM cannot simply digest the input text as is (infinite number of possible input texts)
- We need to map the input text to a sequence of units that the LLM “knows”
- This step is commonly referred to as tokenization, and the units known to the LLM as tokens

This is a tokenization example!

How Do LLMs See Text?

- An LLM cannot simply digest the input text as is (infinite number of possible input texts)
- We need to map the input text to a sequence of units that the LLM “knows”
- This step is commonly referred to as tokenization, and the units known to the LLM as tokens

This is a tokenization example!

`This, _is, _a, _token, ization, _example, !`

Try It Yourself!

- Experiment with different types of input (e.g., different languages, code, etc.)
- What do you observe?
- How would you characterize the generated tokens?

https://platform.openai.com/tokenizer

Types of Tokenization

- Word-level tokenization
- Character-level (or byte-level) tokenization
- Subword-level tokenization

Types of Tokenization

- Word-level tokenization
- Character-level (or byte-level) tokenization
- Subword-level tokenization ← used by most LLMs

Word-Level Tokenization

- Idea: split the input text into a sequence of words
- Widely used in pre-LLM NLP: n-gram models, static word embeddings

Word-Level Tokenization

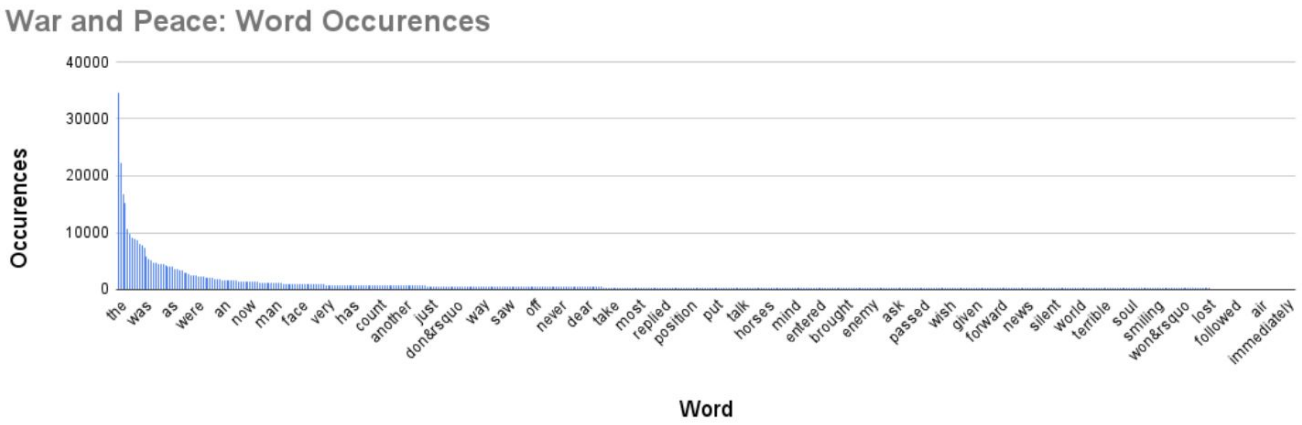
- Idea: split the input text into a sequence of words
- Widely used in pre-LLM NLP: n-gram models, static word embeddings
- Pros:
  - Relatively simple to implement
  - Exact lexical representations

Word-Level Tokenization

- Idea: split the input text into a sequence of words
- Widely used in pre-LLM NLP: n-gram models, static word embeddings
- Pros:
  - Relatively simple to implement
  - Exact lexical representations
- Cons:
  - Large LLM vocabularies are computationally very expensive
  - Heap's law: full coverage of all words impossible → many out-of-vocabulary words

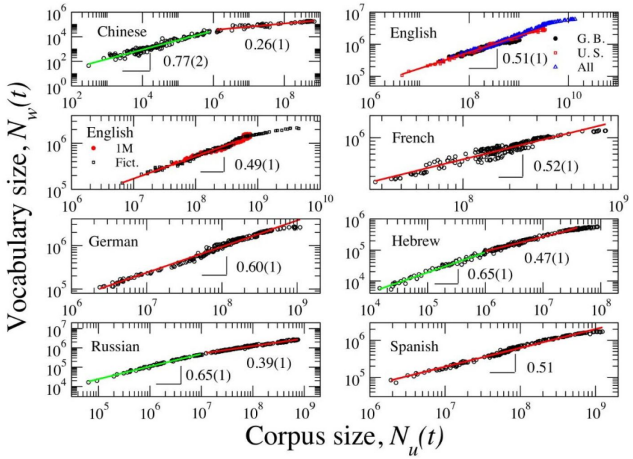
Zipf's Law Recap

- Mathematical form (simplified): $p(w_n) \propto 1/n$
  - p: relative frequency of a word
  - w_n: word at frequency rank n in a text
- Similar statistical relations hold in various natural phenomena (e.g., city sizes)



Heap's Law

- Mathematical form: $V(n) = Kn^{\beta}$
  - V: vocabulary size (i.e., number if distinct word types)
  - n: corpus size (i.e., number of word tokens)
  - K, β: parameters estimated from corpora (typically $10 < K < 100$, $0.4 < \beta < 0.6$)



Temporal Perspective

- New words are constantly created (neologisms, lexicogenesis)

Character-Level Tokenization

- Idea: split the input text into a sequence of characters (or bytes)

Character-Level Tokenization

- Idea: split the input text into a sequence of characters (or bytes)
- Pros:
  - Simple to implement
  - Leads to small token vocabularies

Character-Level Tokenization

- Idea: split the input text into a sequence of characters (or bytes)
- Pros:
  - Simple to implement
  - Leads to small token vocabularies
- Cons:
  - LLM needs to learn lexical representations as a function of characters
  - Difficult to train in practice

Subword-Level Tokenization

Next week!

Questions?

Roadmap for Today

Seminar Introduction ✔

What Is Tokenization, and Why Do LLMs Need It?

LLM Recap (Whiteboard)